

Module 3

Informed Search & Logical Agents

Informed Search Techniques: Heuristic functions, Greedy Best First Search, A* Search. [Ch : 3.5, 3.6]

Logical Agents: Knowledge based agents. The Wumpus world. Logic, Propositional Logic, Reasoning patterns in Propositional Logic. [Ch : 7.1, 7.2, 7.3 & 7.4]

Heuristic:

- ❖ Involving or serving as an aid to learning, discovery, or problem-solving by experimental and especially trial-and-error methods.
- ❖ A core aspect of heuristic search, a problem-solving strategy, involves a form of "trial and error" where different paths or strategies are explored until a solution is found, or a satisfactory one is deemed acceptable.
- ❖ The heuristic function is an estimate, based on domain-specific information that is computable from the current state description, of how close we are to a goal.
- ❖ Specifically, $h(n)$ = estimated cost (or distance) of minimal cost path from n to a goal state.
- ❖ Heuristic technique improves the efficiency of a search process, possibly by sacrificing claims of completeness or optimality.
- ❖ Heuristic is for combinatorial explosion.
- ❖ Optimal solutions are rarely needed.
- All domain knowledge used in the search is encoded in the heuristic function 'h'.
- Heuristic search is an example of a "weak method" because of the limited way that domain-specific information is used to solve the problem.
- Examples:
 - Missionaries and Cannibals: Number of people on starting river bank
 - 8-puzzle: Number of tiles out of place
 - 8-puzzle: Sum of distances each tile is from its goal position
- In general:
 - $h(n) \geq 0$ for all nodes n
 - $h(n) = 0$ implies that n is a goal node
 - $h(n) = \text{infinity}$ implies that n is a dead-end from which a goal cannot be reached
- All the previous searches have been blind searches.
 - They make no use of any knowledge of the problem
 - If we know *something* about the problem, we can usually do much, much better
- Example: 8-puzzle
 - For each piece, figure out how many moves away it is from its goal position, if no other piece were in the way.
 - The total of these gives a measure of distance from goal.
 - This is a *heuristic* measure
- Direct techniques (blind search) are not always possible /to be used for search (they require too much time or memory).
- Weak techniques can be effective if applied correctly on the right kinds of tasks.
 - Typically require domain specific information.

Dr. Yeresime Suresh Ph.D., (NITRR)
 Professor & HOD,
 Dept. of CSE-Artificial Intelligence
 BITM, Ballari-583104.

- ✓ A heuristic is a rule of thumb for deciding which choice might be best
- ✓ There is no general theory for finding heuristics, because every problem is different.
- ✓ Choice of heuristics depends on knowledge of the problem space.

Heuristic Search (informed search)

- A Heuristic is a function that, when applied to a state, returns a number that is an estimate of the merit of the state, with respect to the goal.
- In other words, the heuristic tells us approximately how far the state is from the goal state*.
- Note we said “approximately”. Heuristics might underestimate or overestimate the merit of a state. But for reasons which we will see, heuristics that *only* underestimate are very desirable, and are called admissible.

Heuristic Function:

- Heuristic:
 - Definition: “using rules of thumb to find answers”
- Heuristic function $h(n)$
 - Estimate of (optimal) cost from ‘n’ to goal
 - $h(n) = 0$, if ‘n’ is a goal node
 - Example: straight line distance from n to *Bengaluru*
 - Note that this is not the true state-space distance
 - It is an estimate – actual state-space distance can be higher
 - Provides problem-specific knowledge to the search algorithm

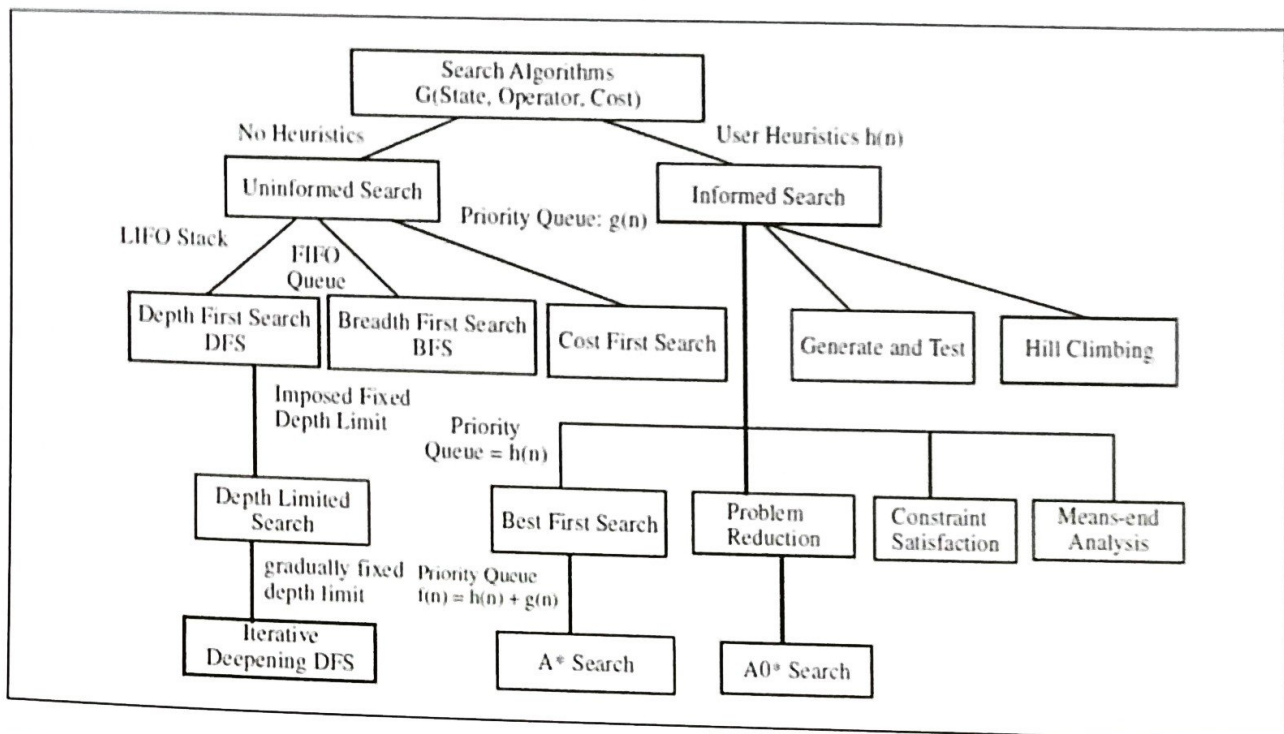
Heuristic Search Techniques:

- ❖ Heuristic techniques are called weak methods, since they are vulnerable to combinatorial explosion. Even then these techniques continue to provide framework into which domain specific knowledge can be placed, either by hand or as a result of learning.
- ❖ There are a variety of search techniques that rely on the estimate provided by a heuristic function.
- ❖ In all cases - the quality (accuracy) of the heuristic is important in real-life application of the technique!

The following are some general purpose control strategies.

1. Generate - and - test
2. Hill climbing
3. Breadth - First search
4. Depth - First search
5. Simulated annealing.
6. Best First Search (A^* search)
7. Problem reduction(AO^* search)
8. Constraint satisfaction
9. Means - ends analysis

Search Algorithms Classification



Best First Search

Depth-first search: Not all competing branches have to be expanded.

Breadth-first search: Not getting trapped on dead-end paths.

Combining the two is to follow a single path at a time, but switch paths whenever some competing path look more promising than the current one.

OR Graphs

- ✓ Each of its branches represents an alternative problem-solving path, we call it as *OR Graph*.
- ✓ At each step of the best-first search process, we select the most promising of the nodes we have generated so far.
 - ✓ This is done by applying an appropriate heuristic function to each of them.
- ✓ We then expand the chosen node by using the rules to generate its successors.
 - ✓ If one of them is a solution, we can quit.
 - ✓ If not, all those new nodes are added to the set of nodes generated so far.
- ✓ Again the most promising node is selected and the process continues.

OPEN: Nodes that have been generated, but have not examined and have had the heuristic function applied to them. This is organized as a priority queue.

CLOSED: Nodes that have already been examined. Whenever a new node is generated, check whether it has been generated before.

- ❖ This procedure is very similar to the procedure for steepest-ascent hill climbing, with two exceptions.
 - ❖ In hill climbing, one move is selected and all the others are rejected, never to be reconsidered.
 - ❖ In best-first search, one move is selected, but the others are kept around so that they can be revisited later if the selected path becomes less promising.
- ❖ Further, the best available state is selected in best-first search, even if that state has a value that is lower than the value of the state that was just explored.
- ❖ This contrasts with hill climbing, which will stop if there are no successor states with better values than the current state.

A* Algorithm - Greedy Best First Search Algorithm

- The best - first search is implemented by an algorithm known as A* algorithm. The algorithm searches a directed graph. Each node will contain a description of the problem state it represents and it will have links to its parent nodes and successor nodes. In addition, it will also indicate how best it is for the search process.
- A* algorithm uses nodes (OPEN) that have been generated, heuristic functions are applied to them, but successors not generated. The list CLOSED contains nodes which have been examined, i.e., their successors generated.
- A heuristic function 'f' estimates the merits of each generated node.
- The function 'g' gives the cost of getting from the initial state to the current node.
- The function 'h' is an estimate of the addition cost of getting from current node to a goal state.
- The function $f (=g+ h)$ gives the cost of getting from the initial state to a goal state via the current node.
- ❖ Algorithm A* (Hart et al., 1968): $f(n) = g(n) + h(n)$

A* Algorithm

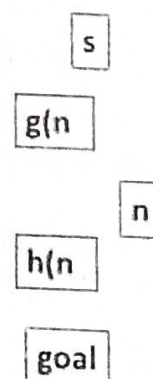
Let $h(n)$ be the heuristic function in a graph. In simple case, let it be the straight line distance SLD from a node to destination.

Let $g(n)$ be the function depending on the distance from source to current node.

A* Search

Thus: $f(n) = g(n) + h(n)$

- Expand node based on estimate of total path cost through node
- Evaluation function $f(n) = g(n) + h(n)$
 - $g(n)$ = cost so far to reach n
 - $h(n)$ = estimated cost from n to goal
 - $f(n)$ = estimated total cost of path through n to goal
- Efficiency of search will depend on quality of heuristic $h(n)$



A* Algorithm Steps

1. Start from source node S, determine all nodes outward from S and queue them.
2. Examine a node from queue (as generated in 1).
 - a. If this node is desired destination node, stop and return success.
 - b. Evaluate $f(n)$ at this node. The node with optimal $f(n)$ gives the next successor, term this node as S.
3. Repeat steps 1 and 2.

THE A* ALGORITHM

1. Start with OPEN containing the initial node.
 - a. $g=0$ and $f' = h'$
 - b. Set CLOSED to empty list.
2. Repeat If OPEN is empty, stop and return failure,
 - a. Else
 - i. pick the BESTNODE on OPEN with lowest f' value and place it on CLOSED
 - b. If BESTNODE is goal state return success and stop
Else
 - c. Generate the successors of BESTNODE.

For each SUCCESSOR do the following:

1. Set SUCCESSOR to point back to BESTNODE
2. Compute $g(\text{SUCCESSOR}) = g(\text{BESTNODE})$ cost of getting from BESTNODE to SUCCESSOR.
3. If SUCCESSOR is the same as any node on OPEN, call that node OLD and add OLD to BESTNODE 's successors. Check $g(\text{OLD})$ and $g(\text{SUCCESSOR})$. If $g(\text{SUCCESSOR})$ is cheaper then reset OLD 's parent link to point to BESTNODE. Update $g(\text{OLD})$ and $f'(\text{OLD})$.
4. If SUCCESSOR was not on OPEN, see if it is on CLOSED. if so call the node CLOSED OLD, and better as earlier and set the parent link and g and f' values appropriately.
5. If SUCCESSOR was not already on earlier OPEN or CLOSED, then put it on OPEN and add it to the list of BESTNODE 's successors.
 1. Compute $f'(\text{SUCCESSOR}) = g(\text{SUCCESSOR}) + h'(\text{SUCCESSOR})$

Recursive Best First Search Algorithm

RBFS is a search algorithm used to find the optimal path to a goal, similar to A*, but with better memory efficiency. It uses recursion and a best-first strategy guided by a heuristic.

Algorithm Steps

1. **Start Search**
Begin with the initial node and an infinite f-cost limit.
2. **Goal Test**
If the current node is the goal, return the solution.
3. **Expand**
Generate all successor nodes from the current node.
4. **Update f-values**
For each successor, update its f-value to ensure it doesn't decrease from its parent's f-cost.
5. **Best Node Selection**
Choose the node with the lowest f-value as the next to explore.
6. **f-limit Check**
If the best node's f-value exceeds the current f-limit, return failure and the new f-limit for backtracking.
7. **Recursive Call**
Recursively call RBFS on the best node, using the lower of the current f-limit or the next best alternative's f-value.
8. **Repeat or Backtrack**
Continue until a solution is found or all paths exceed their limits.

function RECURSIVE-BEST-FIRST-SEARCH(*problem*) **returns** a solution, or failure
return RBFS(*problem*, MAKE-NODE(*problem*.INITIAL-STATE), ∞)

function RBFS(*problem*, *node*, *f_limit*) **returns** a solution, or failure and a new *f*-cost limit
if *problem*.GOAL-TEST(*node*.STATE) **then** **return** SOLUTION(*node*)
successors $\leftarrow$ []
for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**
 add CHILD-NODE(*problem*, *node*, *action*) **into** *successors*
if *successors* is empty **then** **return** failure, ∞
for each *s* **in** *successors* **do** /* update *f* with value from previous search, if any */
 s.f $\leftarrow$ max(*s.g* + *s.h*, *node.f*)
loop do
 best $\leftarrow$ the lowest *f*-value node in *successors*
 if *best.f* > *f_limit* **then** **return** failure, *best.f*
 alternative $\leftarrow$ the second-lowest *f*-value among *successors*
 result, *best.f* $\leftarrow$ RBFS(*problem*, *best*, min(*f_limit*, *alternative*))
 if *result* $\neq$ failure **then** **return** *result*

PROBLEM REDUCTION

- Representing the solution of problems that can be solved by decomposing it into a smaller set of problems.
- This reduction generates ARCS, which we call it has AND arcs.
 - We use AND - OR graph approach.
 - i.e., each/one AND arc points to any number of successor nodes, then
- All of which must be solved in order that the arc should point to a solution.
- In an OR graph, several arcs may emerge from a single node, this indicates a variety of ways in which the original problem might be solved.
- We can find solution to such complex problems by applying Best First Search,

but we are in need of an approach which has the capability to handle the AND arcs appropriately.

But how:

- Algorithm should find a path from the starting node of the graph to a set of nodes representing solution states.,
 - But it is necessary to get more than one solution state...since each arm of an AND arc must lead to its own solution node.

AND - OR Graph Search

- Traverse AND-OR graph, starting from the initial node and follow the current best path.
- Accumulate the set of nodes that are on the best path which have not yet been expanded.
- Pick up one of these unexpanded nodes and expand it.
- Add its successors to the graph and compute f (using only h) for each of them.
- Change the ' f ' estimate of newly expanded node to reflect the new information provided by its successors.
 - Propagate this change backward through the graph to the start.
- Mark the best path which could be different from the current best path.
- Propagation of revised cost in AND-OR graph was not there in A^* .

Problem Reduction through Cost Comprehensibility

- In order to define an algorithm for searching an *AND-OR graph*, one should extract / exploit the value, which is termed as *FUTILITY*.
- If the estimated cost of a solution becomes greater than the value of *futility*, then the search should be abandoned.
- The best way to chose *futility* is that, it corresponds to a threshold value such that any solution with a highest cost is to expensive to be practical, even if it could ever be found.
- Consider a given AND-OR graph...
 - Let us assume that each arc with single successor will have a cost of 1 and each AND arc with multiple successor will have a cost of 1 for each of its components for the sake of simplicity.
 - Here the numbers listed in the circular brackets (19) are estimated cost and the revised costs are enclosed in square brackets [21].
 - Thick lines indicate paths from a given node.

Dr. Yeresime Suresh Ph.D., (NITRKL)

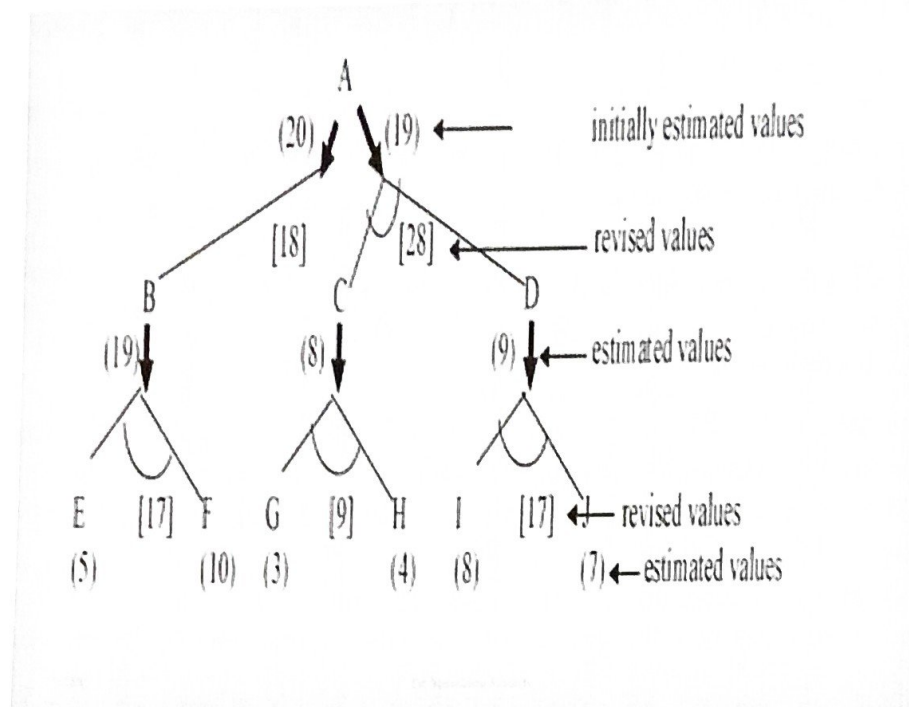
Professor & HOD,
Dept. of CSE-Artificial Intelligence
BITM, Ballari-583104.

Advantages of AO* Star

- ❖ It is an optimal algorithm.
- ❖ It traverses according to the ordering of nodes.
- ❖ It can be used for both OR and AND graph.

Disadvantages of AO* Star

- Sometimes for unsolvable nodes, it can't find the optimal path.
- Its complexity is higher than other algorithms.



1. Initialize the graph to the starting node.
2. Loop until the starting node is labeled *SOLVED* or until its cost goes above *FUTILITY*:
 - (a) Traverse the graph, starting at the initial node and following the current best path, and accumulate the set of nodes that are on that path and have not yet been expanded or labeled as solved.
 - (b) Pick one of these unexpanded nodes and expand it. If there are no successors, assign *FUTILITY* as the value of this node. Otherwise, add its successors to the graph and for each of them compute f' (use only h' and ignore g , for reasons we discuss below). If of any node is 0, mark that node as *SOLVED*.
 - (c) Change the f' estimate of the newly expanded node to reflect the new information provided by its successors. Propagate this change backward through the graph. If any node contains a successor arc whose descendants are all solved, label the node itself as *SOLVED*. At each node that is visited while going up the graph, decide which of its successor arcs is the most promising and mark it as part of the current best path. This may cause the current best path to change. This propagation of revised cost estimates back up the tree was not necessary in the best-first search algorithm because only unexpanded nodes were examined. But now expanded nodes must be reexamined so that the best current path can be selected. Thus it is important that their f' values be the best estimates available.

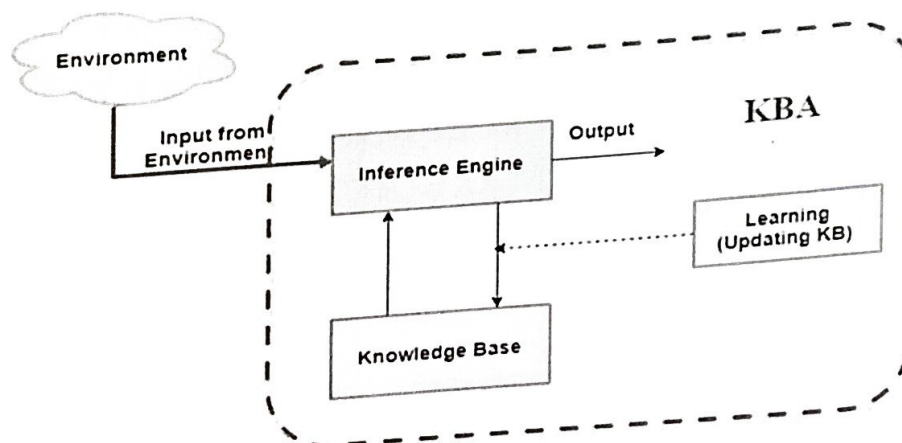
Dr. Yeresime Suresh Ph.D., (NITRKL)
 Professor & HOD,
 Dept. of CSE-Artificial Intelligence
 BITM, Ballari-583104.

LOGICAL AGENTS

In which we design agents that can form representations of a complex world, use a process of inference to derive new representations about the world, and use these new representations to deduce what to do.

KNOWLEDGE - BASED AGENTS

- Knowledge-based agents
 - Intelligent agents need knowledge about the world for making good decisions.
 - A knowledge-based agent includes a knowledge base and an inference system.
- A knowledge base is a set of sentences in a formal language (knowledge representation language) representing facts about the world.
- Inference engine/system derives new sentences from the input and knowledge base.
- A knowledge-based agent must be able to do the following:
 - An agent should be able to represent *states, actions*, etc.
 - An agent should be able to incorporate *new percepts*.
 - An agent can update the *internal representation* of the world.
 - An agent can deduce the *internal representation* of the world.



- The central component of a knowledge-based agent is its **KNOWLEDGE BASE**, or **KB**.
- A knowledge base is a set of sentences. (Here “sentence” is used as a technical term. It is related but not identical to the sentences of English and other natural languages.)
- Each sentence is expressed in a language called a knowledge representation language and represents some assertion about the world. Sometimes we dignify a sentence with the name axiom, when the sentence is taken as given without being derived from other sentences.

Approaches To Build A Knowledge-based Agent:

- There are mainly two approaches to build a knowledge-based agent:
- **Declarative approach:**
 - Tell it what it needs to know.
 - Ask it what to do.
- **Procedural approach:**
 - Encode desired behavior as a program code.

- Which means to write a program that already encodes the desired behavior or agent.
- There must be a way to add new sentences to the knowledge base and a way to query what is known.
 - The standard names for these operations are TELL and ASK, respectively.
- Both operations may involve inference.
- Inference must obey the requirement that when one ASKs a question of the knowledge base, the answer should follow from what has been told (or TELLED) to the knowledge base previously.

Operations of A Knowledge-based Agent

- ❖ Following are three operations which are performed by KBA in order to show the intelligent behavior:
 - TELL: This operation tells the knowledge base what it perceives from the environment.
 - ASK: This operation asks the knowledge base what action it should perform.
 - Perform: It performs the selected action.

LOGICAL AGENTS

- Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

function KB-AGENT(*percept*) *returns an action*

persistent: *KB*, a knowledge base

t, a counter, initially 0, indicating time

TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))

action ← ASK(*KB*, MAKE-ACTION-QUERY(*t*))

TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))

t ← *t* + 1

return *action*

Dr. Yeresime Suresh Ph.D. (AI/ML)
Professor & HOD,
Dept. of CSE-Artificial Intelligence
BITM, Ballari 593104

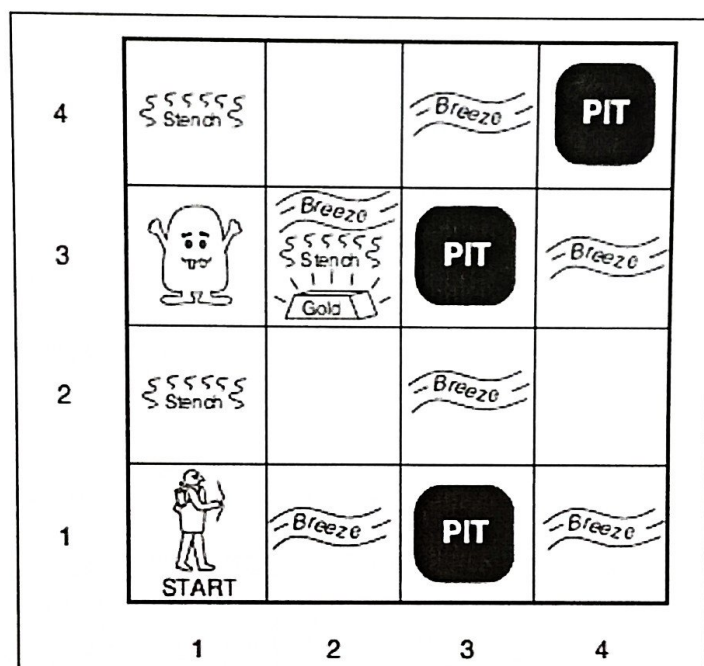
- The KB-Agent algorithm is a model for an intelligent agent that uses a knowledge base (KB) to make decisions. It functions by continuously perceiving its environment and updating its knowledge, allowing it to choose informed actions.
- Designing a rational agent that makes decisions based on logic and inference rather than fixed rules, using an evolving knowledge base to adapt over time is necessary.
- Each time the agent program is called, it does three things.
 - First, it TELLS the knowledge base what it perceives.
 - Second, it ASKs the knowledge base what action it should perform. In the process of answering this query, extensive reasoning may be done about the current state of the world, about the outcomes of possible action sequences, and so on.
 - Third, the agent program TELLS the knowledge base which action was chosen, and the agent executes the action.
- The details of the representation language are hidden inside three functions that implement the interface between the sensors and actuators on one side and the core representation and reasoning system on the other.
 - **MAKE-PERCEPT-SENTENCE** constructs a sentence asserting that the agent perceived

the given percept at the given time.

- **MAKE-ACTION-QUERY** constructs a sentence that asks what action should be done at the current time.
- Finally, **MAKE-ACTION-SENTENCE** constructs a sentence asserting that the chosen action was executed. The details of the inference mechanisms are hidden inside **TELL** & **ASK**.

Wumpus World

- The wumpus world is a cave consisting of rooms connected by passageways. Lurking somewhere in the cave is the terrible wumpus, a beast that eats anyone who enters its room.
- The wumpus can be shot by an agent, but the agent has only one arrow.
- Some rooms contain bottomless pits that will trap anyone who wanders into these rooms (except for the wumpus, which is too big to fall in).



Dr. Yeresime Suresh Ph.D., (NITRKL)
Professor & HOD,
Dept. of CSE-Artificial Intelligence
BITM, Ballari-583104.

- The only mitigating feature of this bleak environment is the possibility of finding a heap of gold.
 - Although the wumpus world is rather tame by modern computer game standards, it illustrates some important points about intelligence.
- A typical Wumpus world: The agent is in the bottom left corner, facing right.
- Precise definition of the task environment is given, by the PEAS description:
 - **Performance measure:** +1000 for climbing out of the cave with the gold, -1000 for falling into a pit or being eaten by the wumpus, -1 for each action taken and -10 for using up the arrow. The game ends either when the agent dies or when the agent climbs out of the cave.
 - **Environment:** A 4x4 grid of rooms. The agent always starts in the square labeled [1,1], facing to the right. The locations of the gold and the wumpus are chosen randomly, with a uniform distribution, from the squares other than the start square. In addition, each square other than the start can be a pit, with probability 0.2.
 - **Actuators:** The agent can move *Forward*, *TurnLeft* by 90°, or *TurnRight* by 90°.
 - The agent dies a miserable death if it enters a square containing a pit or a live wumpus. (It is

- safe, albeit smelly, to enter a square with a dead wumpus.)
- If an agent tries to move forward and bumps into a wall, then the agent does not move.
- The action *Grab* can be used to pick up the gold if it is in the same square as the agent.
- The action *Shoot* can be used to fire an arrow in a straight line in the direction the agent is facing.
- The arrow continues until it either hits (and hence kills) the wumpus or hits a wall.
- The agent has only one arrow, so only the first *Shoot* action has any effect. Finally, the action *Climb* can be used to climb out of the cave, but only from square [1,1].
- **Sensors:** The agent has five sensors, each of which gives a single bit of information:
 - In the square containing the wumpus and in the directly (not diagonally) adjacent squares, the agent will perceive a *Stench*.
 - In the squares directly adjacent to a pit, the agent will perceive a *Breeze*.
 - In the square where the gold is, the agent will perceive a *Glitter*.
 - When an agent walks into a wall, it will perceive a *Bump*.
 - When the wumpus is killed, it emits a woeful *Scream* that can be perceived anywhere in the cave.
- The percepts will be given to the agent program in the form of a list of five symbols; for example, if there is a stench and a breeze, but no glitter, bump, or scream, the agent program will get [Stench, Breeze, None, None, None].
- ✓ Wumpus environment along the various dimensions given: it is discrete, static, and single-agent.
- ✓ It is sequential, because rewards may come only after many actions are taken.
- ✓ It is partially observable, because some aspects of the state are not directly perceivable: the agent's location, the wumpus's state of health, and the availability of an arrow.
- ✓ As for the locations of the pits and the wumpus: we could treat them as unobserved parts of the state that happen to be immutable - in which case, the transition model for the environment is completely known.

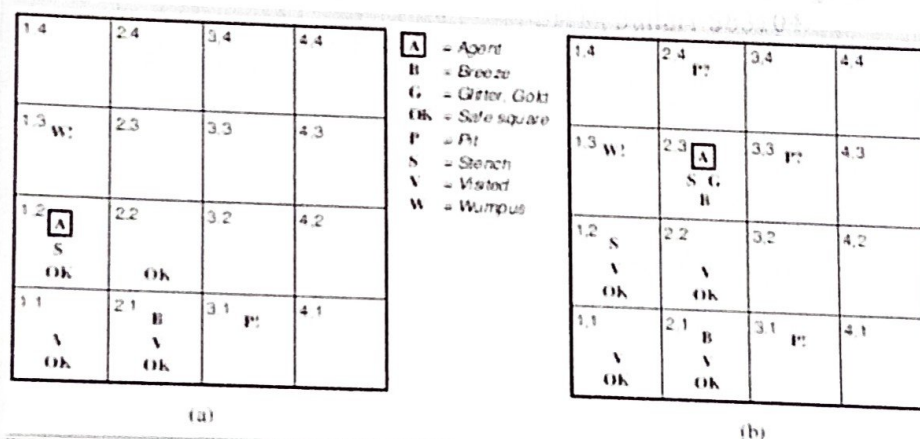


Figure 7.4 Two later stages in the progress of the agent. (a) After the third move, with percept [Stench, None, None, None, None]. (b) After the fifth move, with percept [Stench, Breeze, Glitter, None, None].

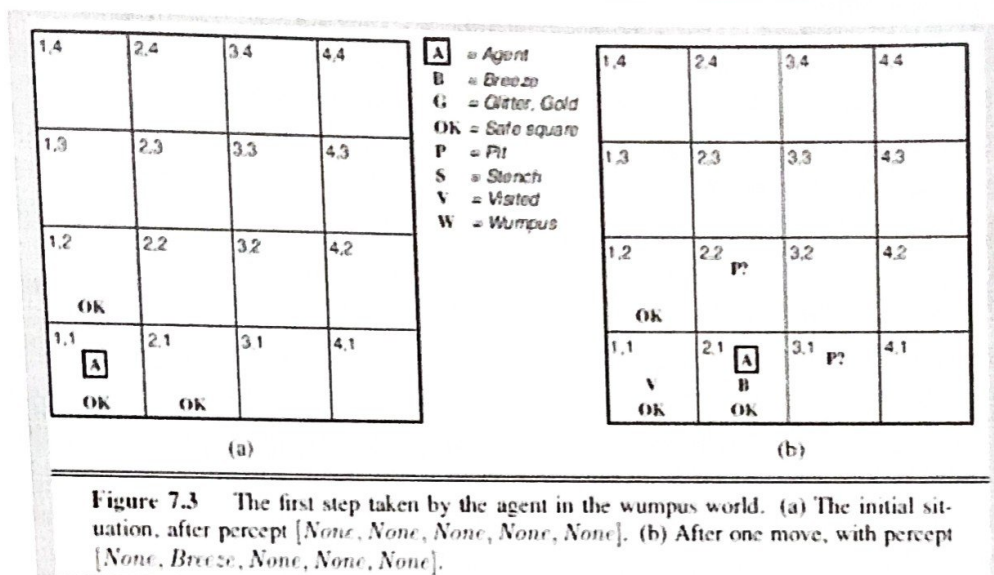


Figure 7.3 The first step taken by the agent in the wumpus world. (a) The initial situation, after percept [None, None, None, None, None]. (b) After one move, with percept [None, Breeze, None, None, None].

LOGIC

What is a Logic?

- A formal language in which knowledge can be expressed
- In Artificial Intelligence, the representation of knowledge is done via Logic.

There are three main components of logic:

- Syntax
 - Rules for constructing legal sentences in the logic
 - Which symbols we can use (English: letters, punctuation)
 - How we are allowed to combine symbols
- Semantics
 - How we interpret (read) sentences in the logic
 - Assigns a meaning to each sentence
- Inference
 - The terms "reasoning" and "inference" are generally used to cover any process by which conclusions are reached

Dr. Yeresime Suresh Ph.D., (NITRKL)
Professor & HOD,
Dept. of CSE-Artificial Intelligence
BITM, Ballari-583104.

Syntax

- The syntax of propositional logic defines the allowable sentences. The atomic sentences consist of a single proposition symbol. Each such symbol stands for a proposition that can be true or false.
- There are two proposition symbols with fixed meanings:
 - True is the always-true proposition and
 - False is the always-false proposition.

Complex sentences are constructed from simpler sentences, using parentheses and logical connectives. There are five connectives in common use:

- $\neg$ (not). A sentence such as $\neg W_{1,3}$ is called the **negation** of $W_{1,3}$. A **literal** is either an atomic sentence (a **positive literal**) or a negated atomic sentence (a **negative literal**).
- $\wedge$ (and). A sentence whose main connective is $\wedge$, such as $W_{1,3} \wedge P_{3,1}$, is called a **conjunction**; its parts are the **conjuncts**. (The $\wedge$ looks like an "A" for "And.")
- $\vee$ (or). A sentence using $\vee$, such as $(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$, is a **disjunction** of the **disjuncts** $(W_{1,3} \wedge P_{3,1})$ and $W_{2,2}$. (Historically, the $\vee$ comes from the Latin "vel," which means "or." For most people, it is easier to remember $\vee$ as an upside-down $\wedge$.)
- $\Rightarrow$ (implies). A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called an **implication** (or **conditional**). Its **premise** or **antecedent** is $(W_{1,3} \wedge P_{3,1})$, and its **conclusion** or **consequent** is $\neg W_{2,2}$. Implications are also known as **rules** or **if-then** statements. The implication symbol is sometimes written in other books as $\supset$ or $\rightarrow$.
- $\Leftrightarrow$ (if and only if). The sentence $W_{1,3} \Leftrightarrow \neg W_{2,2}$ is a **biconditional**. Some other books write this as $\equiv$.

Dr. Yeresime Suresh Ph.D., (NITRR)
Professor & HOD,
Dept. of CSE-Artificial Intelligence
BITM, Ballari-583104.

<i>Sentence</i>	$\rightarrow$	<i>AtomicSentence</i> <i>ComplexSentence</i>
<i>AtomicSentence</i>	$\rightarrow$	<i>True</i> <i>False</i> <i>P</i> <i>Q</i> <i>R</i> ...
<i>ComplexSentence</i>	$\rightarrow$	(<i>Sentence</i>) [<i>Sentence</i>]
		$\neg$ <i>Sentence</i>
		<i>Sentence</i> $\wedge$ <i>Sentence</i>
		<i>Sentence</i> $\vee$ <i>Sentence</i>
		<i>Sentence</i> $\Rightarrow$ <i>Sentence</i>
		<i>Sentence</i> $\Leftrightarrow$ <i>Sentence</i>

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

- The BNF grammar by itself is ambiguous; a sentence with several operators can be parsed by the grammar in multiple ways. To eliminate the ambiguity we define a precedence for each operator. The "not" operator ($\neg$) has the highest precedence, which means that in the sentence $\neg A \wedge B$ the $\neg$ binds most tightly, giving us the equivalent of $(\neg A) \wedge B$ rather than $\neg(A \wedge B)$.
 - (The notation for ordinary arithmetic is the same: $-2+4$ is 2, not -6 .)
- When in doubt, use parentheses to make sure of the right interpretation. Square brackets mean the same thing as parentheses; the choice of square brackets or parentheses is solely to make it easier for a human to read a sentence.

Semantics

- The semantics defines the rules for determining the truth of a sentence with respect to a particular model.
- In propositional logic, a model simply fixes the truth value -true or false - for every proposition symbol. For example, if the sentences in the knowledge base make use of the proposition symbols $P_{1,2}$, $P_{2,2}$, and $P_{3,1}$, then one possible model is":

$$m_1 = \{ P_{1,2} = false, P_{2,2} = false, P_{3,1} = true \}$$

- With three proposition symbols, there are $2^3=8$ possible models.
- However, that the models are purely mathematical objects with no necessary connection to wumpus worlds. $P_{1,2}$ is just a symbol; it might mean "there is a pit in [1,2]" or "I'm in Paris today and tomorrow."

- The semantics for propositional logic must specify how to compute the truth value of *any* sentence, given a model. This is done recursively.
- All sentences are constructed from atomic sentences and the five connectives; therefore, we need to specify how to compute the truth of atomic sentences and how to compute the truth of sentences formed with each of the five connectives. Atomic sentences are easy:
- True is true in every model and False is false in every model.
- The truth value of every other proposition symbol must be specified directly in the model. For example, in the model m_1 given earlier, P_{12} is false.

For complex sentences, we have five rules, which hold for any sub-sentences P and Q in any model m (here “iff” means “if and only if”):

- $\neg P$ is true iff P is false in m .
- $P \wedge Q$ is true iff both P and Q are true in m .
- $P \vee Q$ is true iff either P or Q is true in m .
- $P \Rightarrow Q$ is true unless P is true and Q is false in m .
- $P \Leftrightarrow Q$ is true iff P and Q are both true or both false in m .

The rules can also be expressed with truth tables that specify the truth value of a complex sentence for each possible assignment of truth values to its components. Truth tables for the five connectives are:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Figure 7.8 Truth tables for the five logical connectives. To use the table to compute, for example, the value of $P \vee Q$ when P is true and Q is false, first look on the left for the row where P is *true* and Q is *false* (the third row). Then look in that row under the $P \vee Q$ column to see the result: *true*.

A simple knowledge base : Wumpus World

$P_{x,y}$ is true if there is a pit in $[x, y]$.

$W_{x,y}$ is true if there is a wumpus in $[x, y]$, dead or alive.

$B_{x,y}$ is true if the agent perceives a breeze in $[x, y]$.

$S_{x,y}$ is true if the agent perceives a stench in $[x, y]$.

The sentences we write will suffice to derive $\neg P_{1,2}$ (there is no pit in $[1,2]$), as was done informally in Section 7.3. We label each sentence R_i so that we can refer to them:

- There is no pit in $[1,1]$:

$$R_1 : \neg P_{1,1}.$$

- A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square; for now, we include just the relevant squares:

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}).$$

- The preceding sentences are true in all wumpus worlds. Now we include the breeze percepts for the first two squares visited in the specific world the agent is in, leading up to the situation in Figure 7.3(b).

$$R_4 : \neg B_{1,1}.$$

$$R_5 : B_{2,1}.$$

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	true
false	true	false	false	false	true	false	true	true	true	true	true	true
false	true	false	false	true	false	false	true	false	false	true	true	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	true	true	false	true	false

Figure 7.9 A truth table constructed for the knowledge base given in the text. KB is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in $[1,2]$. On the other hand, there might (or might not) be a pit in $[2,2]$.

- A truth-table enumeration algorithm for deciding propositional entailment.
- (TT stands for truth table.) PL-TRUE? returns *true* if a sentence holds within a model.
- The variable *model* represents a partial model—an assignment to some of the symbols.

The keyword “and” is used here as a logical operation on its two arguments, returning true or false.


```

function TT-ENTAILS?(KB,  $\alpha$ ) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          $\alpha$ , the query, a sentence in propositional logic

  symbols  $\leftarrow$  a list of the proposition symbols in KB and  $\alpha$ 
  return TT-CHECK-ALL(KB,  $\alpha$ , symbols, { })

function TT-CHECK-ALL(KB,  $\alpha$ , symbols, model) returns true or false
  if EMPTY?(symbols) then
    if PL-TRUE?(KB, model) then return PL-TRUE?( $\alpha$ , model)
    else return true // when KB is false, always return true
  else do
    P  $\leftarrow$  FIRST(symbols)
    rest  $\leftarrow$  REST(symbols)
    return (TT-CHECK-ALL(KB,  $\alpha$ , rest, model  $\cup$  {P = true})
           and
           TT-CHECK-ALL(KB,  $\alpha$ , rest, model  $\cup$  {P = false}))

```

Standard logical equivalences

```

( $\alpha \wedge \beta$ )  $\equiv$  ( $\beta \wedge \alpha$ )  commutativity of  $\wedge$ 
( $\alpha \vee \beta$ )  $\equiv$  ( $\beta \vee \alpha$ )  commutativity of  $\vee$ 
(( $\alpha \wedge \beta$ )  $\wedge \gamma$ )  $\equiv$  ( $\alpha \wedge (\beta \wedge \gamma)$ )  associativity of  $\wedge$ 
(( $\alpha \vee \beta$ )  $\vee \gamma$ )  $\equiv$  ( $\alpha \vee (\beta \vee \gamma)$ )  associativity of  $\vee$ 
 $\neg(\neg\alpha)$   $\equiv$   $\alpha$   double-negation elimination
( $\alpha \Rightarrow \beta$ )  $\equiv$  ( $\neg\beta \Rightarrow \neg\alpha$ )  contraposition
( $\alpha \Rightarrow \beta$ )  $\equiv$  ( $\neg\alpha \vee \beta$ )  implication elimination
( $\alpha \Leftrightarrow \beta$ )  $\equiv$  (( $\alpha \Rightarrow \beta$ )  $\wedge$  ( $\beta \Rightarrow \alpha$ ))  biconditional elimination
 $\neg(\alpha \wedge \beta)$   $\equiv$  ( $\neg\alpha \vee \neg\beta$ )  De Morgan
 $\neg(\alpha \vee \beta)$   $\equiv$  ( $\neg\alpha \wedge \neg\beta$ )  De Morgan
( $\alpha \wedge (\beta \vee \gamma)$ )  $\equiv$  (( $\alpha \wedge \beta$ )  $\vee$  ( $\alpha \wedge \gamma$ ))  distributivity of  $\wedge$  over  $\vee$ 
( $\alpha \vee (\beta \wedge \gamma)$ )  $\equiv$  (( $\alpha \vee \beta$ )  $\wedge$  ( $\alpha \vee \gamma$ ))  distributivity of  $\vee$  over  $\wedge$ 

```

Summary

- Best-first search is general search where the minimum-cost nodes (according to some measure) are expanded first.
- Greedy search uses minimal estimated cost $h(n)$ to the goal state as measure. This reduces the search time, but the algorithm is neither complete nor optimal.
- A* search combines BFS & DFS,
 - $f(n) = g(n) + h(n)$.
 - A* handles state repetitions and $h(n)$ never overestimates.
 - A* is complete and optimal, but space complexity is high.
 - The time complexity depends on the quality of the heuristic function.
- Intelligent agents need knowledge about the world in order to reach good decisions.

Principles of Artificial Intelligence

- Knowledge is contained in agents in the form of sentences in a knowledge representation language that are stored in a knowledge base.
- A knowledge-based agent is composed of a knowledge base and an inference mechanism.
- It operates by storing sentences about the world in its knowledge base, using the inference mechanism to infer new sentences, and using these sentences to decide what action to take.
- A representation language is defined by its syntax, which specifies the structure of sentences, and its semantics, which defines the truth of each sentence in each possible world or model.
- The relationship of entailment between sentences is crucial to our understanding of reasoning. A sentence α entails another sentence β if β is true in all worlds where α is true. Equivalent definitions include the validity of the sentence $\alpha \Rightarrow \beta$ and the un-satisfiability of the sentence $\alpha \wedge \neg\beta$.
- Inference is the process of deriving new sentences from old ones. Sound inference algorithms derive only sentences that are entailed; complete algorithms derive all sentences that are entailed.
- Propositional logic is a simple language consisting of proposition symbols and logical connectives. It can handle propositions that are known true, known false, or completely unknown.
- Propositional logic does not scale to environments of unbounded size because it lacks the expressive power to deal concisely with time, space, and universal patterns of relationships among objects.